VOLUME 03: TRACK 2 - OMNICODE LANGUAGE & STANDARD LIBRARY MANUAL

Chapter 20: Track 2 - Standard Library Modules

Core Logic, Vector Mathematics, High-Precision Arithmetic & Memory Management

> ***Witness:*** *1 Corinthians 14:40 (KJV) - Let all things be done decently and in order*

SYSTEM TRACK
<TRACK 2> : {OmniCode Language & Standard Library}

CHAPTER IDENTIFIER
CH 20

NOTEBOOKLM WORD COUNT
3,672 words

CATALOGED CHAPTER FILES
8 files (674 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"How do the standard library modules export pure algebraic invariants without side effects?"

CHAPTER MANIFEST & INDEX (8 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/kernel/boot.omni**	OMNI	114	679	4.9
2	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/kernel/boot.tape**	TAPE	50	487	3.4
3	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/arithmetic.omni**	OMNI	90	416	2.8
4	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/cell.tape**	TAPE	17	133	0.9
5	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/logic.omni**	OMNI	89	404	2.9
6	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/memory.omni**	OMNI	135	703	5.2
7	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/memory.tape**	TAPE	28	174	1.2
8	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/vectors.omni**	OMNI	151	676	5.1

<TRACK 2> : {OmniCode Language & Standard Library}/lib/kernel/boot.omni OMNI 114 LINES 679 WORDS 4.9 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Kernel Architecture: Genesis Bare-Metal OS Specification )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the bare-metal machine initialization contract:
#   - Establishes the 64-bit silicon envelope hosting the 32-trit kernel word.
#   - Mounts the 4096-cell Root_RAM storehouse on Track 0 [KERNEL].
#   - Writes the boot banner to the memory-mapped VGA text buffer at 0x000B8000.
#   - Initializes the 4-directional Memory Reel spindle hierarchy.
# ================================================================================

Module < Kernel.GenesisBoot > ( MilleniumOS Genesis bare-metal kernel initialization ) {
    version: < "1.0.0" >,
    system: < MilleniumOS.Kernel >,
    semantics: < BareMetalBootstrap :: BalancedTernaryALU :: MMIOConsole >
}

# 1. Kernel Entity: < KERNEL >
Kernel < InvariantNoun > ( Master Operating System Supervisor ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+Kernel" >,
    concrete_form: < "Kernel-" >,
    authority_channel: < "Track 0 [KERNEL]" >,
    machine_effect: < "hardware_root_supervisor" >
}

# 2. Storehouse Entity: < ROOT_RAM >
RootRAM < InvariantNoun > ( Physical Silicon Memory Storehouse ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+RootRAM" >,
    concrete_form: < "RootRAM-" >,
    capacity_cells: < 4096 >,
    machine_effect: < "physical_ram_allocator" >
}

# 3. Video Console Entity: < VGA_SCREEN >
ScreenVGA < InvariantNoun > ( Bare-Metal Video Text Buffer at 0x000B8000 ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+ScreenVGA" >,
    concrete_form: < "ScreenVGA-" >,
    mmio_base: < 0x000B8000 >,
    geometry: < "80x25 characters" >,
    machine_effect: < "vga_text_buffer" >
}

# 4. Serial Transducer Entity: < SERIAL_COM1 >
SerialCOM1 < InvariantNoun > ( UART Serial Hardware Port at 0x03F8 ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+SerialCOM1" >,
    concrete_form: < "SerialCOM1-" >,
    io_port: < 0x03F8 >,
    baud_rate: < 115200 >,
    machine_effect: < "hardware_input_transducer" >
}

# 5. Silicon Register Bank Entity: < REGISTERS >
RegisterBank < InvariantNoun > ( 13 Named Silicon Registers and Transducers ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+RegisterBank" >,
    concrete_form: < "RegisterBank-" >,
    capacity_slots: < 13 >,
    registers: < [ HEAD HOME SPOOL IT SELF CALLER OTHER STATE TURN BOUND FROM INTO DISCARD ] >,
    machine_effect: < "silicon_register_allocation" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Kernel : InvariantNoun ::
        RootRAM : InvariantNoun ::
        ScreenVGA : InvariantNoun ::
```

```
        SerialCOM1 : InvariantNoun ::
        RegisterBank : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Roll 1: Allocate root RAM storehouse and arm 13 Named Silicon Registers
    ram_and_regs [ Root_RAM [ 4096 ] :: Register_Bank [ HEAD HOME SPOOL IT SELF CALLER OTHER STATE TURN BOUND FROM INTO DISCARD ] :: FROM ::@ 0x03F8 :: INTO ::@ 0x000B8000 ] ( Root memory storehouse and 13 named silicon registers armed )

    # Roll 2: Prove ALU equilibrium invariant and calibrate Quad-State accumulator loop
    alu_and_loop [ ALU_Equilibrium [ (+1) plus (-1) <=> 0 ] :: Accumulator_Loop [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN } ! [ { IT } > { BOUND } ] AWAIT for [ IT` ] :: HALT WHEN [ IT` ] != even ] ] ( Invariant arithmetic equilibrium and quad-state loop calibration )

    # Roll 3: Carve protected Kernel and Terminal partitions via v-- and evaluate contextual bounds
    partition_carve [ Root_RAM v-- Kernel_Partition [ 1024 ] :: Root_RAM v-- Terminal_Partition [ 512 ] :: Bounds_Check [ { 1024 } < { 4096 } ] ] ( Partition storehouses via v-- and bounds evaluation )

    # Roll 4: Inscribe initial landmark and boot banner to MMIO VGA video buffer
    landmark_and_banner [ 0 --> Kernel_Partition @ 0 :: 'MILLENIUMOS: OMNICODE 32-TRIT OS ONLINE' --> Screen_VGA ::@ 0x000B8000 ] ( Landmark and screen banner )

    # Roll 5: Commit genesis boot
    commit_genesis [ commit_status [ PROCEED ] ] ( Genesis Boot sealed in PROCEED )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        ram_and_regs = yes ::
        alu_and_loop = yes ::
        partition_carve = yes ::
        landmark_and_banner = yes ::
        commit_genesis = yes
    ],
    commit_status [ PROCEED ] ( Genesis Boot verified and sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/kernel/boot.tape TAPE 50 LINES 487 WORDS 3.4 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Tape Manifest, Kernel Scheduling & Genesis Machine Covenant )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes cold boot sequence on Track 0 [KERNEL]:
#   - Senses physical Root_RAM storehouse and binds serial COM1 situated at 0x03F8 via ::@.
#   - Proves balanced ternary ALU equilibrium before allocating mutable memory partitions.
#   - Carves Kernel_Partition (1024 cells) and Terminal_Partition (512 cells) via v--, verifying [ { 1024 } < { 4096 } ].
#   - Establishes preemptive turn-taking (~! & PREEMPT) and awakens VGA console at 0x000B8000.
#   - Seals genesis boot in affirmative PROCEED.
# ================================================================================

[Tape.GenesisBoot] : WHO {MilleniumOS_Kernel} WHAT {SystemGenesis} --> BOOT {Hardware}
    WHEN {PowerOn}
    WHERE {ram} = [ 4096 ]
    WHY ( Initialize balanced ternary MilleniumOS kernel and diagnostic nervous system )
    HOW [ +1 : PROCEED ]
    :: < ancient_landmark > ( Proverbs 22:28 - 'Remove not the ancient landmark' )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environmental Preconditions, Memory Partition Geometry & Storehouse Bounds )
# ================================================================================

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Sequential Instruction Rolls Lowered to Silicon Machine Words )
# ================================================================================

# Roll 1: Sense physical memory and arm 13 Named Silicon Registers with hardware transducers
roll_1 [ Kernel --> SENSE {Root_RAM [ 4096 ]} :: ARM {Registers [ HEAD HOME SPOOL IT SELF CALLER OTHER STATE TURN BOUND FROM INTO DISCARD ]} :: FROM ::@ 0x03F8 :: INTO ::@ 0x000B8000 ] ( Sense physical memory and arm 13 named silicon registers with hardware transducers )

# Roll 2: Prove ALU equilibrium invariant and calibrate Quad-State accumulator loop
roll_2 [ Kernel --> PROVE {ALU_Equilibrium} [ (+1) plus (-1) <=> 0 ] :: LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN } ! [ { IT } > { BOUND } ] AWAIT for [ IT` ] :: HALT WHEN [ IT` ] != even ] ] ( Invariant proof of balanced ternary arithmetic and quad-state accumulator calibration )

# Roll 3: Carve protected kernel and terminal partitions via v-- and evaluate contextual bounds
roll_3 [ Root_RAM v-- Kernel_Partition [ 1024 ] :: Root_RAM v-- Terminal_Partition [ 512 ] :: Bounds_Check [ { 1024 } < { 4096 } ] ] ( Partition storehouse via v-- and evaluate bounds in context )

# Roll 4: Yield compute on timer tick and awaken console HUD with MMIO boot banner
roll_4 [ Kernel_Compute ~! [ Timer_Tick --> PREEMPT {Background_Task} ] :: 'MILLENIUMOS: OMNICODE 32-TRIT OS ONLINE' --> Screen_VGA ::@ 0x000B8000 :: Kernel --> AWAKEN {Terminal_Studio} ] ( Preempt compute on timer interrupt and awaken screen banner )

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Invariant Verification Proofs & Covenant Commit Seal )
# ================================================================================

# Roll 5: Genesis Boot Complete
roll_5 [ commit_status [ PROCEED ] ] ( MilleniumOS Genesis Boot Complete )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/arithmetic.omni OMNI 90 LINES 416 WORDS 2.8 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Core Library Manifest: Invariant Arithmetic Macros & Opcodes )
# ========================================================================
Module < Core.Arithmetic > ( Invariant arithmetic macros linked to CPU and LLVM opcodes ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < MacroInvariants :: HardwareOpcodes :: SVOTranspilation >
}

# Macro Invariant: < ADD >
ADD < InvariantMacro > ( Binary Addition ) {
    micro: < "plus" >,
    x86_64: < "add" >,
    riscv: < "add" >,
    llvm_ir: < "add i64" >,
    c_operator: < "+" >
}

# Macro Invariant: < SUB >
SUB < InvariantMacro > ( Binary Subtraction ) {
    micro: < "minus" >,
    x86_64: < "sub" >,
    riscv: < "sub" >,
    llvm_ir: < "sub i64" >,
    c_operator: < "-" >
}

# Macro Invariant: < MUL >
MUL < InvariantMacro > ( Binary Multiplication ) {
    micro: < "times" >,
    x86_64: < "imul" >,
    riscv: < "mul" >,
    llvm_ir: < "mul i64" >,
    c_operator: < "*" >
}

# Macro Invariant: < DIV >
DIV < InvariantMacro > ( Binary Division ) {
    micro: < "div" >,
    x86_64: < "idiv" >,
    riscv: < "div" >,
    llvm_ir: < "sdiv i64" >,
    c_operator: < "/" >
}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ( Preconditions & Macro Registry Setup )
# ========================================================================
Opening {
    requires: [ Hardware.ALU :: IntegerRegisters.64 ],
    preconditions: [ +1 ]
}

test_x [ 40 ] ( Sample operand X )
test_y [ 2 ]  ( Sample operand Y )

# ========================================================================
# BLOCK 3: BODY BLOCK
# ( SVO Descriptions Lowered to Invariant Macro Opcodes )
# ========================================================================
Body [
    # 1. Addition: Micro 'plus' lowers to Macro < ADD >
    sum_result [ test_x plus test_y ] ( 40 plus 2 = 42 )

    # 2. Subtraction: Micro 'minus' lowers to Macro < SUB >
    diff_result [ sum_result minus test_y ] ( 42 minus 2 = 40 )

    # 3. Multiplication: Micro 'times' lowers to Macro < MUL >
    prod_result [ test_y times 10 ] ( 2 times 10 = 20 )

    # 4. Division: Micro 'div' lowers to Macro < DIV >
    quot_result [ prod_result div 2 ] ( 20 div 2 = 10 )
```

```
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Verification & Macro Status Commit )
# ================================================================================
Closing {
    verify: [
        sum_result = 42 ::
        diff_result = 40 ::
        prod_result = 20 ::
        quot_result = 10
    ],
    commit_status [ PROCEED ] ( Arithmetic library verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/cell.tape TAPE 17 LINES 133 WORDS 0.9 KB

```
# ================================================================================
# TAPE HEAD: THE CELL DEFINITION COVENANT
# ================================================================================
[Cell] : A unit of measure --> WHERE {unit} = [ 8 bit --> 5 trit --> 8 bit ] :: < measure > ( Fundamental unit of memory
)

# ================================================================================
# SUBORDINATE ROLLS: CELL DENSE ENCODING & REVERSIBLE TRANSDUCTION
# ================================================================================
# Roll 1: Transduce byte 42 into 5 trits and store in cell 0
roll_1 [ 42 --> Cell_0 @ 0 ] ( Inscribe radix-3 packed value into cell 0 )

# Roll 2: Verify cell 0 holds 42 within invariant bounds
roll_2 [ Cell_0 = 42 ! [ +1 ] ] ( Verify dense byte recovery )

# Roll 3: Transduce and seal with affirmative commit
roll_3 [ commit_status [ PROCEED ] ] ( Cell covenant proven )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/logic.omni OMNI 89 LINES 404 WORDS 2.9 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Core Library Manifest: Invariant Quad-State Macros & OS Control Opcodes )
# ================================================================================
Module < Core.Logic > ( Invariant logic macros linked to CPU execution and kernel control ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < MacroInvariants :: KernelPhysics :: QuadStateOpcodes >
}

# Macro Invariant: < PROCEED >
PROCEED_MACRO < InvariantMacro > ( Advance Execution & Commit State Frame ) {
    micro: < "yes" >,
    radix3: < 1 >,
    x86_64: < "jmp .Lnext" >,
    riscv: < "j .Lnext" >,
    c_status: < "0" >,
    os_action: < "commit_transaction_and_advance" >
}

# Macro Invariant: < ROLLBACK >
ROLLBACK_MACRO < InvariantMacro > ( Retroactive Unwind & Snapshot Restore ) {
    micro: < "no" >,
    radix3: < -1 >,
    x86_64: < "mov rsp, rbp; pop rbp; ret" >,
    riscv: < "mv sp, s0; ld s0, 0(sp); ret" >,
    c_status: < "-1" >,
    os_action: < "unwind_transaction_frame" >
}

# Macro Invariant: < AWAIT >
AWAIT_MACRO < InvariantMacro > ( CPU Idle / Asynchronous Interrupt Standby ) {
    micro: < "ok" >,
    radix3: < 0 >,
    x86_64: < "hlt" >,
    riscv: < "wfi" >,
    arm64: < "wfi" >,
    os_action: < "yield_cpu_to_scheduler" >
}

# Macro Invariant: < HALT >
HALT_MACRO < InvariantMacro > ( Unrecoverable Boundary Trap / Panic ) {
    micro: < "void" >,
    radix3: < NULL >,
    x86_64: < "ud2" >,
    riscv: < "ebreak" >,
    c_status: < "abort()" >,
    os_action: < "kernel_panic_trap" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Kernel Preconditions & Invariant Verification )
# ================================================================================
Opening {
    requires: [ CPU.InterruptControl :: ProcessTable.Active ],
    preconditions: [ +1 ]
}

status_flag [ yes ] ( Active execution thread status )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Logic Invariants Mapped to OS Physics )
# ================================================================================
Body [
    # 1. Monotonic Commit Test
    commit_test [ status_flag = PROCEED ] ( 'yes' triggers monotonic forward commit )

    # 2. Inversion Symmetry Test
    negate_test [ -- status_flag = ROLLBACK ] ( Inversion of 'yes' triggers ROLLBACK )

    # 3. Equilibrium Balance Test
    balance_test [ ok = AWAIT ] ( 'ok' maps to AWAIT / HLT / WFI )
```

```
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Verification & Status Commit )
# ================================================================================
Closing {
    verify: [
        commit_test = yes ::
        negate_test = yes ::
        balance_test = yes
    ],
    commit_status [ PROCEED ] ( Logic core sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/memory.omni OMNI 135 LINES 703 WORDS 5.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Core Library Manifest: Foundational Storage Nouns & Memory Vessel Invariants )
# ================================================================================
Module < Core.Memory > ( Foundational storage nouns, bounded vessels, and hardware transducers ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < StorageNouns :: BoundedVessels :: TransducedCells :: BoundaryPhysics >
}

# 1. Atomic Data Unit: < TRIT >
TRIT_NOUN < InvariantNoun > ( The Atomic Radix-3 Polarity Unit ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_tier: < "Tier 1: Atomic Unit" >,
    permitted_states: < "Neg_T_Zero_0_Pos_1" >,
    machine_effect: < "radix3_polarity_register" >
}

# 2. Atomic Data Unit: < CELL >
CELL_NOUN < InvariantNoun > ( The Atomic 5-Trit / 8-Bit Quad-State Memory Vessel ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_tier: < "Tier 1: Atomic Unit" >,
    physical_footprint: < "1 physical byte in RAM (243 values + 13 control states)" >,
    machine_effect: < "dense_byte_trit_cell" >
}

# 3. Storage Container: < VESSEL >
VESSEL_NOUN < InvariantNoun > ( Bounded Sequential Container of Memory Cells ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_tier: < "Tier 2: Storage Container" >,
    invariant_law: < "Proverbs 22:28 (Remove not the ancient landmark)" >,
    machine_effect: < "contiguous_bounded_buffer" >
}

# 4. Storage Container: < STOREHOUSE >
STOREHOUSE_NOUN < InvariantNoun > ( Root System Memory Domain & Reservoir ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_tier: < "Tier 2: Storage Container" >,
    machine_effect: < "root_page_reservoir" >
}

# 5. Storage Container: < PARTITION >
PARTITION_NOUN < InvariantNoun > ( Entrusted Bounded Sub-Region of a Storehouse ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_tier: < "Tier 2: Storage Container" >,
    machine_effect: < "delegated_task_slice" >
}

# 6. Coordinate: < OFFSET >
OFFSET_NOUN < InvariantNoun > ( Relative Zero-Based Coordinate within a Vessel ) {
    grammar_primary: < "Preposition" >,
    grammar_secondary: < "Noun" >,
    ontological_tier: < "Tier 3: Spatial Coordinate" >,
    machine_effect: < "memory_address_displacement" >
}

# 7. Measure: < CAPACITY >
CAPACITY_NOUN < InvariantNoun > ( Maximum Allocated Cell Limit of a Vessel ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_tier: < "Tier 4: Measure" >,
    machine_effect: < "boundary_limit_ceiling" >
}

# 8. Measure: < COUNT >
COUNT_NOUN < InvariantNoun > ( Number of Actively Written or Populated Cells ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_tier: < "Tier 4: Measure" >,
```

```
    machine_effect: < "active_population_count" >
}

# 9. Substance: < VALUE >
VALUE_NOUN < InvariantNoun > ( The Numeric or Symbolic Payload Stored in a Cell ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_tier: < "Tier 5: Substance" >,
    machine_effect: < "cell_payload_register" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Preconditions & Vessel Context Setup )
# ================================================================================
Opening {
    requires: [ MemoryAligned :: QuadStateTransducer :: BoundaryPhysics ],
    preconditions: [ +1 ]
}

# Declare the system Storehouse
Main_Storehouse [ 1024 ] { Fixed, Bounded } ( System memory reservoir of 1024 cells )

# Grant an entrusted Partition carved directly from Main_Storehouse via v--
Main_Storehouse v-- Task_Partition [ 256 ] ( Entrusted process partition of 256 cells )

# Initial test data
sample_val [ 42 ] ( Natural number 42 to store )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Memory Operations: Forward Store, Feedback Read, and Bounds Verification )
# ================================================================================
Body [
    # 1. Forward Causality: Store natural value 42 into cell 0 of partition
    step_store [ sample_val --> Task_Partition[0] ] ( Deposit value into cell 0 )

    # 2. Feedback Retrieval: Reflect value back from cell 0
    step_fetch [ read_val <-- Task_Partition[0] ] ( Retrieve value from cell 0 )

    # 3. State Inscription: Mark cell 1 with affirmative PROCEED state
    step_state [ PROCEED --> Task_Partition[1] ] ( Inscribe PROCEED into cell 1 )

    # 4. Invariant Verification: Confirm within capacity
    step_bound [ Task_Partition[0] ! [ +1 ] ] ( Enforce capacity caveat )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Integrity Verification & Invariant Seal )
# ================================================================================
Closing {
    verify: [
        step_store = +1 ::
        step_fetch = +1 ::
        step_state = +1 ::
        step_bound = +1
    ],
    commit_status [ PROCEED ] ( Memory vessel core sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/memory.tape TAPE 28 LINES 174 WORDS 1.2 KB

```
# ================================================================================
# TAPE HEAD: MEMORY RESERVOIR & PARTITIONING COVENANT
# ================================================================================
[Tape.MemoryManager] : WHO {Kernel} WHAT {Storehouse} --> DISPATCH {PartitionEngine}
    WHEN {Boot}
    WHERE {partition} = [ 1024 ]
    WHY ( Allocate and protect physical memory domains )
    HOW [ +1 : PROCEED ]
    :: < covenant > ( verified )

# ================================================================================
# SUBORDINATE ROLLS: PARTITIONING & CELL INDEXING
# ================================================================================
# Roll 1: Allocate root storehouse of 1024 cells
roll_1 [ Root_Storehouse [ 1024 ] ] ( Root memory domain )

# Roll 2: Carve task partition of 256 cells via v--
roll_2 [ Root_Storehouse v-- Task_Partition [ 256 ] ] ( Isolated task memory slice )

# Roll 3: Inscribe value 99 into Task_Partition at cell 0
roll_3 [ 99 --> Task_Partition @ 0 ] ( Direct cell write )

# Roll 4: Inseparable key & value dual binding
roll_4 [ Key&Value [ 1 & 99 ] ] ( Inseparable dual binding )

# Roll 5: Seal memory tape
roll_5 [ commit_status [ PROCEED ] ] ( Memory tape verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/vectors.omni OMNI 151 LINES 676 WORDS 5.1 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Core Library Manifest: Invariant Directional Macros & Vector Opcodes )
# ========================================================================
Module < Core.Vectors > ( Invariant vector macros linked to machine dispatch and branching ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < MacroInvariants :: VectorDispatch :: CPUBranching >
}

# Macro Invariant: < ACT_UPON > (Right: Forward Causation)
ACT_UPON_MACRO < InvariantMacro > ( Forward State Dispatch & Mutation ) {
    canonical: < "-->" >,
    shorthand: < "->" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "+X (Temporal Causality)" >,
    machine_effect: < "forward_pipeline_commit" >
}

# Macro Invariant: < ACT_AGAIN > (Left: Retroaction / Rollback)
ACT_AGAIN_MACRO < InvariantMacro > ( Retroactive Feedback & State Rollback ) {
    canonical: < "<--" >,
    shorthand: < "<-" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "-X (Feedback / Rewind)" >,
    machine_effect: < "restore_snapshot_branch" >
}

# Macro Invariant: < BUILDS_TO > (Up: Synthesis / Abstraction)
BUILDS_TO_MACRO < InvariantMacro > ( Layering Up into Abstract Domain ) {
    canonical: < "--^" >,
    shorthand: < "-^" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "+Z (Scale / Synthesis)" >,
    machine_effect: < "type_promotion_and_framing" >
}

# Macro Invariant: < BREAKS_DOWN_FROM > (Down: Decomposition / Analysis)
BREAKS_DOWN_FROM_MACRO < InvariantMacro > ( Layering Down into Hardware Primitive ) {
    canonical: < "--v" >,
    shorthand: < "-v" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "-Z (Lowering / Destructuring)" >,
    machine_effect: < "register_unpack_and_lowering" >
}

# Scale Modulations (Major, Minor, Great, Small)
MAJOR_MACRO < InvariantMacro > ( Major Ascending Scale Modulation ) {
    canonical: < "/-^" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adverb" >,
    axis: < "+Scale Major" >,
    machine_effect: < "major_scale_elevation" >
}

MINOR_MACRO < InvariantMacro > ( Minor Descending Scale Modulation ) {
    canonical: < "\\-v" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adverb" >,
    axis: < "-Scale Minor" >,
    machine_effect: < "minor_scale_lowering" >
}

GREAT_MACRO < InvariantMacro > ( Great Ascending Scale Modulation ) {
    canonical: < "\\-^" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adverb" >,
    axis: < "+Scale Great" >,
    machine_effect: < "great_scale_elevation" >
}
```

```
SMALL_MACRO < InvariantMacro > ( Small Descending Scale Modulation ) {
    canonical: < "/-v" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adverb" >,
    axis: < "-Scale Small" >,
    machine_effect: < "small_scale_lowering" >
}

# 2D Quadrant Spatial Vectors
UPPER_RIGHT_MACRO < InvariantMacro > ( Quadrant I Spatial Vector: +X, +Y ) {
    canonical: < "/^" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "+X, +Y (Upper Right)" >,
    machine_effect: < "quadrant_1_spatial_vector" >
}

LOWER_RIGHT_MACRO < InvariantMacro > ( Quadrant IV Spatial Vector: +X, -Y ) {
    canonical: < "\\v" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "+X, -Y (Lower Right)" >,
    machine_effect: < "quadrant_4_spatial_vector" >
}

UPPER_LEFT_MACRO < InvariantMacro > ( Quadrant II Spatial Vector: -X, +Y ) {
    canonical: < "^\\" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "-X, +Y (Upper Left)" >,
    machine_effect: < "quadrant_2_spatial_vector" >
}

LOWER_LEFT_MACRO < InvariantMacro > ( Quadrant III Spatial Vector: -X, -Y ) {
    canonical: < "v/" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "-X, -Y (Lower Left)" >,
    machine_effect: < "quadrant_3_spatial_vector" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Preconditions & Vector Context )
# ================================================================================
Opening {
    requires: [ Pipeline.DirectExecution ],
    preconditions: [ +1 ]
}

x_pos [ 1 ] ( Initial X-axis coordinate )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Directional Macro Operations )
# ================================================================================
Body [
    # 1. Forward Causality: Acts upon x_pos, advancing to 10
    step_forward [ x_pos --> 10 ] ( Advance forward along +X )

    # 2. Feedback: Reflects back upon x_pos
    step_feedback [ x_pos <-- 1 ] ( Retroactive check along -X )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Verification & Commit )
# ================================================================================
Closing {
    verify: [
        step_forward = 10 ::
        step_feedback = 10
    ],
    commit_status [ PROCEED ] ( Vector core sealed )
}
```